

Comms-Denied Autonomy Hackathon

Technical Guide & Primer

This guide covers the challenge, the system you'll build against, and how to approach building a solution.

Logistics & how to prepare

Teams of **up to 5**. The event runs **about 6 hours**, and you write all code during it. **No pre-written code**. You download the simulator on the day (it's designed to run on a typical laptop), and **you get no starter kit**, so you'll wire up the control and video connections yourself.

Because you can't build ahead, prepare by getting fluent with the moving parts rather than by writing code: practice sending Offboard setpoints over MAVLink, capturing and processing a video stream, and running an object detector and an optical-flow / VIO library.

1. The Challenge

Build an **autonomous agent** that flies a simulated quadcopter through a course of visual puzzles and navigation segments with **zero human intervention**. The scenario models a communications- and GPS-denied environment: no remote piloting and no reliable GPS. Your agent perceives through the onboard camera and telemetry, decides, and commands the drone.

The one rule that matters most

You are graded on a **held-out scene** you have never seen. Cue colors, object identities, counts, and exact geometry will differ from anything you develop against; only the course *structure* stays fixed. Hard-coding a path will fail. Build something that **generalizes**.

2. System & Interfaces

The simulator runs on Unreal Engine 5 with **PX4 as the flight stack** (software-in-the-loop), pre-configured by the organizers. You interact through two channels, and you're free to use any language or libraries to do so.

- **Control & telemetry.** Command the drone by sending the flight controller *offboard* setpoints (position, velocity, attitude, or rate/thrust) and read telemetry (IMU, attitude, altitude, etc.) back. The controller speaks the standard MAVLink protocol; the client side is your choice.
- **Video feed.** The simulator streams a live onboard camera feed (RGB, possibly with additional views such as depth/segmentation) as a video stream (most likely over WebRTC), separate from the control channel. Exact access details come with the day-of materials.

Control modes

PX4 runs the low-level stabilization on the flight controller itself; what changes between modes is where the high-level commands come from. In **Offboard mode**, PX4 follows setpoints your program streams in real time. **This is the mode you will use**. Its built-in autonomous alternative, **Mission mode**, flies a waypoint plan uploaded in advance, which is not usable here: you can't know the course ahead of time, and there's no GPS in Hard mode to navigate by.

("Onboard" is not a separate PX4 flight mode; it means a companion computer mounted on the vehicle, which still commands PX4 through Offboard. In the simulator, your program is that external source.) Within Offboard you choose *what* to stream: position or velocity setpoints (which need a position estimate; see below) or attitude/rate/thrust setpoints (which don't, but leave all position-level control to you).

Easy vs. Hard mode

Develop with ground truth, deploy without, mirroring real robotics. **Grading is on Hard only**; Easy is a sandbox to validate your sensing and control.

	Easy (sandbox)	Hard (graded, held-out)
Position source	GPS available to the flight controller	No GPS
Offboard control	Position/velocity setpoints work out of the box	Work once your own estimate reaches the controller
Use it to	Validate perception & control against truth	Prove autonomy with no external positioning

No GPS: read this carefully

The flight controller needs a position estimate before it will accept *position/velocity* commands. In Hard mode there is no GPS, so you must produce your *own* estimate of where the drone is (from optical flow, visual-inertial methods, or other onboard sensing) and feed it to the controller; otherwise it won't hold or move to a position. The controller is pre-configured to accept an external estimate, so this works out of the box once you supply one. You don't need to build this estimator from scratch. Integrate an existing optical-flow or visual-inertial (VIO) library. Getting it working is the core of Hard mode, so plan for it early.

3. What the Course Will Demand

Without giving away the specifics, expect your agent to need to:

- Read visual cues from the scene and navigate by them.
- **Carry information forward.** Something you see early may determine an action much later, and the original cue may be long out of view by then. Reacting to the current frame alone will not be enough.
- Perceive attributes and quantities of objects, not only whether they are present.
- Navigate stretches where no external cue or positioning is available, relying on its own sense of motion.
- Single out one specific target among similar ones and act on it.

4. Building Your Agent

Most solutions are loops run many times per second: **perceive** (latest frame + telemetry) → **understand** (what's there, what the segment wants) → **decide** (pick a target) → **act** (send a setpoint) → **log**. Your design is how you fill each step and how fast you run it. Treat these as components you build and test independently:

- **Perception.** Pixels/depth → meaning: detect cues, count objects, read the final targets. Anything from classical CV (color/edge/blob, optical flow) to learned detectors to vision-language models.
- **State estimation.** Without GPS, fuse IMU + barometer + visual motion into pose/velocity. In Hard mode this is also the estimate you feed to the flight controller to unlock position/velocity control. Integrate an existing optical-flow / VIO library rather than writing one from scratch.
- **Decision / task logic.** Identify the current segment and choose the next action, using a state machine, search/planning, or an LLM/VLM reasoning step. Keep the goal explicit so you can log why.
- **Control.** Convert intent into smooth setpoints. Offboard control needs a **continuous setpoint stream** or the controller drops out of offboard mode. Prefer velocity/position setpoints; tune for stability before speed.
- **Logging & dashboard.** A required deliverable and your best debugging tool: timestamped logs linking each decision to the visual cue that triggered it, plus annotated frames and reasoning traces. Build it early.

5. Evaluation & Submission

Final runs are on a fresh/modified scene. The organizers release the full scoring rubric with the day-of materials; expect **task completion and speed** to be central.

Submit a **screen recording** of your drone completing the course, with the simulator's built-in **timer visible in frame**.

Run-time internet and cloud calls are **allowed**, so hosted vision-language models are viable. Managing their latency is your problem. Per the brief, also be ready to show *how* your agent perceives and decides (e.g. annotated frames, reasoning traces).

6. Tips & Common Pitfalls

- **Don't overfit.** Test against changes you make yourself; the held-out scene differs in every detail except structure.
- **Fly first, think later.** Get stable hover + waypoint motion working before adding any intelligence.
- **Don't rely on Easy-mode GPS.** If your agent only flies with GPS, it will faceplant on the graded Hard run. Build and test the no-GPS path.
- **Mind offboard quirks.** Stream setpoints without pause (PX4 needs them faster than ~2 Hz) and keep your position estimate flowing; if either stops, the flight controller drops out of offboard mode.
- **Watch latency.** A 2-second-per-frame model may be too slow to fly. Decouple slow “thinking” from fast “reflexes.”

7. Suggested First Steps

Once you have the simulator on the day, build up in this order:

1. Connect to the control interface, read telemetry, and pull a frame from the video feed.
2. Arm, take off, and hold a position/velocity setpoint in Easy mode (with GPS).
3. Add one perception capability (detect one cue) and steer toward it; wrap it in the perceive–decide–act loop with logging.
4. Stand up your own position estimate and feed it to the controller; re-fly the same task in Hard mode (no GPS).

Resources

MAVLink Protocol: mavlink.io

PX4 offboard control: docs.px4.io/.../offboard

flying without GPS / using an external position estimate: [docs.px4.io \(EKF2\)](http://docs.px4.io (EKF2))

Glossary

SITL (software-in-the-loop): the flight controller's software runs as a program on your computer, with the simulator standing in for the drone and its sensors.

Flight stack: the autopilot software (here, PX4) that stabilizes the drone and carries out commands.

Offboard mode: the PX4 mode in which the drone follows setpoints your program streams in real time; the mode you will use.

Setpoint: one target command sent to the controller (a position, velocity, attitude, or thrust value).

MAVLink: the standard protocol for talking to the flight controller (sending setpoints, reading telemetry).

Telemetry: live state data from the drone, e.g. IMU, attitude, and altitude.

EKF2: PX4's state estimator; fuses sensor data into a position/attitude estimate. It needs a position source (GPS, or your own estimate) before it will allow position control.

Optical flow: estimating motion from how the image shifts between consecutive frames.

VIO (visual-inertial odometry): estimating the drone's motion and pose by combining camera and IMU data; one way to replace GPS.

Held-out scene: the unseen scene your agent is graded on, different from anything you developed against.